🏛 GLA UNIVERSITY, MATHURA — DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

HPC Masterclass | Day 3 Pre-Assessment: Foundations of Deep Learning Systems, Hardware & Cluster Computing

Target Audience: 7th Semester B.Tech CSE (HPC Elective) | **Total Questions**: 50 | **Duration**: 60 Minutes

📋 Answer Key & Forensic Solution Guide (Instructor Copy)

Q#	Correct Answer	Cognitive Bloom Level	Core Concept Focus
01	B	`Understand`	In standard deep learning training, what is the pr...
02	B	`Analyze`	What is the key difference between Eager Execution...
03	A	`Knowledge`	In TensorFlow 2.x, which Python decorator transfor...
04	A	`Understand`	Why is mini-batch Stochastic Gradient Descent (SGD...
05	B	`Understand`	How does the Adam optimizer calculate parameter up...
06	B	`Knowledge`	What is the primary motivation for applying Batch ...
07	B	`Application`	What happens to the spatial dimensions (height and...
08	B	`Analyze`	What causes the 'vanishing gradient problem' in de...
09	B	`Knowledge`	What role does the 'Dropout' regularization techni...
10	B	`Understand`	In deep learning frameworks, why must intermediate...
11	A	`Knowledge`	How many bits are used for the exponent and mantis...
12	B	`Analyze`	What is the structural bit-level difference betwee...
13	A	`Understand`	Why does mixed-precision training (using FP16 or B...
14	A	`Knowledge`	In the context of post-training quantization, what...
15	B	`Understand`	What is the mathematical definition of 'Arithmetic...
16	B	`Analyze`	According to the Roofline Model, what fundamental ...
17	C	`Analyze`	Which of the following neural network operations p...
18	A	`Understand`	What is the purpose of 'Loss Scaling' when perform...
19	B	`Analyze`	What is the primary trade-off encountered when app...
20	B	`Knowledge`	What does a dynamic range underflow error mean dur...
21	A	`Knowledge`	What is the typical access latency of CPU L1 Data ...

Q#	Correct Answer	Cognitive Bloom Level	Core Concept Focus
22	C	`Knowledge`	What is the standard hardware cache line size on m...
23	A	`Understand`	What does SIMD stand for in computer architecture,...
24	C	`Knowledge`	How wide are Intel AVX-512 vector registers (ZMM0 ...
25	A	`Analyze`	Why is the PCIe bus frequently a major performance...
26	B	`Understand`	In a Non-Uniform Memory Access (NUMA) multi-socket...
27	B	`Knowledge`	What is High Bandwidth Memory (HBM2e / HBM3), and ...
28	B	`Understand`	What is the purpose of an input data pipeline pref...
29	B	`Understand`	What is a 'memory leak' in Python deep learning co...
30	B	`Analyze`	Why is 'data layout transformation' (e.g., convert...
31	B	`Understand`	In distributed deep learning, what is the core con...
32	B	`Analyze`	When is 'Model Parallelism' (Tensor Parallelism or...
33	C	`Knowledge`	In the MPI standard, what does the collective oper...
34	B	`Understand`	What is the key functional difference between 'Red...
35	B	`Analyze`	What is the primary challenge of Asynchronous SGD ...
36	B	`Understand`	What is the 'straggler problem' in synchronous dis...
37	A	`Analyze`	Why is gradient bucketing (or gradient fusion) uti...
38	B	`Knowledge`	What is the difference between latency and bandwid...
39	B	`Understand`	What is Remote Direct Memory Access (RDMA), and wh...
40	C	`Application`	In a distributed cluster of 8 nodes with 8 GPUs ea...
41	B	`Understand`	What is the primary role of a Workload Manager (Jo...
42	B	`Knowledge`	In a Slurm cluster, what is the role of the 'slurm...
43	B	`Knowledge`	Which command is used to submit a non-interactive ...
44	A	`Understand`	What is the difference between 'srun' and 'sbatch'...
45	B	`Knowledge`	Which Slurm directive specifies the maximum allowa...
46	C	`Application`	What does the symbol '%j' represent in the Slurm d...
47	B	`Knowledge`	What does the command 'sinfo' display in a Slurm c...
48	B	`Analyze`	Why is training deep learning models on a shared N...
49	B	`Understand`	What is the purpose of the 'Environment Modules' s...
50	B	`Application`	If a user needs to cancel a queued or running Slur...

🔬 Complete Questions with Detailed Technical Rationale

Question 01

In standard deep learning training, what is the primary computational purpose of the backward pass (backpropagation)?

- **(a)** To compute predicted outputs from input features.
- **(b) [CORRECT]** To evaluate gradients of the scalar loss with respect to all trainable parameters via the chain rule of calculus.
- **(c)** To convert 32-bit floating-point tensors into 8-bit integers.
- **(d)** To schedule worker threads on available CPU cores.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: Backpropagation computes the gradient of the loss function with respect to every layer parameter by systematically applying the chain rule from the final output layer backwards to the inputs.

Question 02

What is the key difference between Eager Execution and Static Computational Graphs in deep learning frameworks like TensorFlow?

- **(a)** Eager execution requires writing all code in C++, whereas static graphs use Python.
- **(b) [CORRECT]** Eager execution evaluates operations immediately and imperatively, while static graphs construct a declarative symbolic DAG (Directed Acyclic Graph) before execution.
- **(c)** Static graphs do not support GPU acceleration.
- **(d)** Eager execution completely disables backpropagation.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Analyze`
> **Architectural Rationale**: In eager execution, operations are executed line-by-line returning concrete values immediately, aiding debugging. Static graphs capture the computation as a symbolic DAG, enabling compiler optimizations like dead-code elimination, kernel fusion, and constant folding.

Question 03

In TensorFlow 2.x, which Python decorator transforms a dynamic Python function into a high-performance compiled static TensorFlow graph?

- **(a) [CORRECT]** @tf.function
- **(b)** @tf.compile_graph
- **(c)** @tf.eager_mode
- **(d)** @tf.static_run

> **Correct Answer**: **(A)**
> **Bloom's Cognitive Level**: `Knowledge`
> **Architectural Rationale**: '@tf.function' builds a callable TensorFlow graph by tracing the Python code with AutoGraph, enabling backend graph optimization passes and hardware compilation.

Question 04

Why is mini-batch Stochastic Gradient Descent (SGD) preferred over full-batch Gradient Descent when training on large datasets?

- **(a) [CORRECT]** Full-batch GD requires loading the entire dataset into memory at once, which exceeds system RAM, and offers fewer gradient updates per epoch.
- **(b)** Full-batch GD is mathematically incapable of minimizing convex loss functions.
- **(c)** Mini-batch SGD disables all multi-threading on modern processors.
- **(d)** Mini-batch SGD guarantees zero variance in gradient estimates.

> **Correct Answer**: **(A)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: Full-batch GD requires computing gradients over the entire dataset before making a single weight update, which is memory-prohibitive for large datasets. Mini-batching fits in GPU/accelerator RAM and provides frequent updates with helpful stochastic regularization.

Question 05

How does the Adam optimizer calculate parameter updates compared to basic Stochastic Gradient Descent (SGD)?

- **(a)** Adam sets all weights to zero at the end of each epoch.
- **(b) [CORRECT]** Adam maintains per-parameter adaptive learning rates by tracking both first moments (mean) and uncentered second moments (variance) of the gradients.
- **(c)** Adam computes second-order Hessian matrices directly using numerical differentiation.
- **(d)** Adam updates only the bias terms and ignores weight matrices.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: Adam computes individual adaptive learning rates for different parameters from estimates of first and second moments of the gradients, incorporating bias corrections.

Question 06

What is the primary motivation for applying Batch Normalization layers in deep neural network architectures?

- **(a)** To replace all activation functions with identity operations.
- **(b) [CORRECT]** To stabilize optimization by normalizing layer inputs across the batch dimension to zero mean and unit variance, mitigating internal covariate shift.
- **(c)** To reduce the physical storage size of the dataset on disk.
- **(d)** To encrypt model weights during training.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Knowledge`
> **Architectural Rationale**: Batch Normalization normalizes activations across the batch dimension during training, smoothing the optimization landscape and permitting higher learning rates.

Question 07

What happens to the spatial dimensions (height and width) of a 32×32 image when processed by a 3×3 2D convolution with stride 1 and 'VALID' padding?

- **(a)** They expand to 34×34.
- **(b) [CORRECT]** They shrink to 30×30.
- **(c)** They remain exactly 32×32.
- **(d)** They drop to 16×16.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Application`
> **Architectural Rationale**: Formula for output dimension with valid padding is $O = \lfloor (W - K)/S \rfloor + 1$. Here: $(32 - 3)/1 + 1 = 29 + 1 = 30$. The output dimensions are 30×30.

Question 08

What causes the 'vanishing gradient problem' in deep neural networks using Sigmoid or Tanh activations?

- **(a)** The hardware runs out of floating-point registers.
- **(b) [CORRECT]** The derivative of Sigmoid is bounded between 0 and 0.25, causing repeated multiplication across many layers in backpropagation to exponentially decay gradients toward zero.
- **(c)** The learning rate is automatically multiplied by 100 after each layer.
- **(d)** The input dataset has too many training features.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Analyze`
> **Architectural Rationale**: Because $\sigma'(z) \leq 0.25$, multiplying gradients through many layers during backpropagation causes exponential decay, preventing early layers from updating their weights.

Question 09

What role does the 'Dropout' regularization technique play during deep learning training?

- **(a)** It deletes corrupt image files from the training directory.
- **(b) [CORRECT]** It randomly zeroes out a fraction p of neuron activations during each forward pass, preventing complex co-adaptations between neurons.
- **(c)** It drops slow worker nodes from the distributed cluster.
- **(d)** It reduces the precision of all model parameters to 8-bit integers.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Knowledge`
> **Architectural Rationale**: Dropout prevents overfitting by stochastically deactivating a subset of neurons at each iteration, forcing the network to learn robust, distributed representations.

Question 10

In deep learning frameworks, why must intermediate layer activations computed during the forward pass be cached in memory?

- **(a)** To allow the operating system to verify user credentials.
- **(b) [CORRECT]** Because the partial derivatives computed during backpropagation require the activation values from the forward pass as multipliers.
- **(c)** To prevent the GPU from entering sleep mode.
- **(d)** To display real-time training images on the monitor.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: By the chain rule (e.g., $\frac{\partial L}{\partial W} = \frac{\partial L}{\partial z} \cdot a^T$), the gradient with respect to weights directly depends on the forward activations a, requiring them to be retained until backward pass completion.

Question 11

How many bits are used for the exponent and mantissa (fraction) in standard IEEE 754 single-precision floating point (FP32)?

- **(a) [CORRECT]** 1 sign bit, 8 exponent bits, 23 mantissa bits.
- **(b)** 1 sign bit, 5 exponent bits, 10 mantissa bits.
- **(c)** 1 sign bit, 11 exponent bits, 52 mantissa bits.
- **(d)** 1 sign bit, 8 exponent bits, 7 mantissa bits.

> **Correct Answer**: **(A)**
> **Bloom's Cognitive Level**: `Knowledge`
> **Architectural Rationale**: FP32 consists of 1 sign bit, 8 exponent bits (dynamic range approx 10^{-38} to 10^{38}), and 23 mantissa bits (giving ~7 decimal digits of precision).

Question 12

What is the structural bit-level difference between standard FP16 and Google/Intel Bfloat16 (BF16)?

- **(a)** FP16 uses 16 bits, while BF16 uses 32 bits.
- **(b) [CORRECT]** FP16 has 5 exponent bits and 10 mantissa bits, whereas BF16 has 8 exponent bits and 7 mantissa bits (matching FP32's dynamic range).
- **(c)** BF16 cannot represent negative numbers.
- **(d)** FP16 is only used on CPUs, while BF16 is only used on hard drives.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Analyze`
> **Architectural Rationale**: BF16 preserves the 8-bit exponent of FP32 while truncating the fraction to 7 bits. This gives BF16 the exact same dynamic range as FP32, preventing underflow/overflow during gradient updates without requiring complex loss scaling.

Question 13

Why does mixed-precision training (using FP16 or BF16 for matrix operations alongside FP32 master weights) provide major speedups on modern HPC accelerators?

- **(a) [CORRECT]** It halves memory bandwidth pressure, doubles tensor core compute throughput, and reduces cache footprints by 50%.
- **(b)** It eliminates the need for backpropagation entirely.
- **(c)** It automatically converts deep neural networks into decision trees.
- **(d)** It bypasses the operating system kernel network stack.

> **Correct Answer**: **(A)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: 16-bit representations require half the memory bandwidth of FP32 (2 bytes vs 4 bytes), allow twice as many operands per cache line, and utilize specialized hardware tensor engines (e.g., Intel AMX, NVIDIA Tensor Cores) running at 2x to 8x higher FLOPS.

Question 14

In the context of post-training quantization, what is INT8 quantization?

- **(a) [CORRECT]** Compressing weights and activations from 32-bit floats into 8-bit signed integers using scale and zero-point parameters.
- **(b)** Deleting 8 out of every 10 layers in a deep neural network.
- **(c)** Rounding all floating-point numbers to zero.
- **(d)** Running neural networks on an 8-bit Intel 8085 microprocessor.

> **Correct Answer**: **(A)**
> **Bloom's Cognitive Level**: `Knowledge`
> **Architectural Rationale**: INT8 quantization maps a continuous range of floating-point values $[r_{min}, r_{max}]$ to 8-bit integers $[-128, 127]$ using affine transformation: $q = \text{round}(r/S) + Z$, slashing memory footprint by $4\times$ and using integer vector ALUs.

Question 15

What is the mathematical definition of 'Arithmetic Intensity' (Operational Intensity) in the Roofline Model?

- **(a)** Total Wall Clock Time / Total CPU Cores
- **(b) [CORRECT]** Total Floating-Point Operations (FLOPs) / Total Bytes transferred from main memory (FLOPs/Byte)
- **(c)** Dataset Size in Megabytes / Batch Size
- **(d)** Network Packet Size / Clock Frequency

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: Arithmetic Intensity is measured in FLOPs per Byte ($I = \text{FLOPs}/\text{Bytes}$). It dictates whether an algorithm is memory-bandwidth bound ($I < I_{ridge}$) or compute bound ($I > I_{ridge}$) on a specific processor.

Question 16

According to the Roofline Model, what fundamental hardware characteristic limits the maximum throughput of an operation with very low arithmetic intensity?

- **(a)** Peak theoretical FLOPS of the vector ALUs.
- **(b) [CORRECT]** Memory bandwidth (DRAM or HBM transfer rate in GB/s or TB/s).
- **(c)** Hard drive seek latency.
- **(d)** Compiler optimization flags.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Analyze`
> **Architectural Rationale**: When an operation performs very few arithmetic calculations per byte loaded (e.g., elementwise vector addition or ReLU), execution units spend most cycles stalled waiting for data from memory. Performance is strictly bounded by memory bandwidth.

Question 17

Which of the following neural network operations possesses the HIGHEST arithmetic intensity?

- **(a)** Elementwise Vector Addition ($y = a + b$)
- **(b)** ReLU Activation ($y = \max(0, x)$)
- **(c) [CORRECT]** General Matrix Multiplication (GEMM, $C = A \times B$) with large square dimensions ($N > 2048$)
- **(d)** Layer Normalization

> **Correct Answer**: **(C)**
> **Bloom's Cognitive Level**: `Analyze`
> **Architectural Rationale**: GEMM performs $2N^3$ FLOPs on $3N^2$ elements (data size $O(N^2)$), yielding an arithmetic intensity of $O(N)$ FLOPs/byte, which grows with matrix size and becomes highly compute-bound.

Question 18

What is the purpose of 'Loss Scaling' when performing mixed-precision training with IEEE FP16?

- **(a) [CORRECT]** To artificially multiply the model loss by a scale factor to prevent small gradient values from underflowing to zero in 16-bit representation.
- **(b)** To divide the loss to ensure training finishes in fewer epochs.
- **(c)** To convert the loss value into an integer percentage.
- **(d)** To normalize the dataset images to mean zero.

> **Correct Answer**: **(A)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: FP16 has a narrow dynamic range (10^{-5} minimum normal positive number). Small gradient values would underflow to zero. Multiplying the loss by a factor (e.g., 1024 or 65536) shifts gradients into the representable range of FP16 before backpropagation.

Question 19

What is the primary trade-off encountered when applying weight pruning to a deep neural network?

- **(a)** Pruning guarantees zero loss in accuracy regardless of sparsity percentage.
- **(b) [CORRECT]** Unstructured pruning creates irregular sparse matrices that require specialized sparse accelerators to achieve actual wall-clock speedups compared to dense GEMMs.
- **(c)** Pruning doubles the memory footprint of the model on disk.
- **(d)** Pruning converts the neural network into an uncompressed audio waveform.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Analyze`
> **Architectural Rationale**: While unstructured pruning removes non-critical weights, random zeros lead to irregular memory access patterns. Standard dense tensor hardware (BLAS/cuBLAS) cannot easily accelerate unstructured sparsity without dedicated sparse tensor cores (e.g., 2:4 structured sparsity).

Question 20

What does a dynamic range underflow error mean during floating-point matrix multiplication?

- **(a)** The result is larger than the maximum positive number representable, becoming positive infinity.
- **(b) [CORRECT]** The magnitude of a non-zero number is smaller than the smallest subnormal representable number, causing it to be flushed to zero.
- **(c)** The CPU registers have overheated.
- **(d)** The input tensor contains negative numbers.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Knowledge`
> **Architectural Rationale**: Underflow occurs when a calculation produces a number closer to zero than the precision format can represent ($< 2^{-126}$ for FP32 or $< 2^{-14}$ for FP16), resulting in loss of precision or rounding to zero.

Question 21

What is the typical access latency of CPU L1 Data Cache compared to Main Memory (DRAM)?

- **(a) [CORRECT]** L1 cache is ~1 nanosecond (4-5 cycles), while DRAM is ~60-80 nanoseconds (200+ cycles).
- **(b)** L1 cache and DRAM have identical latency.
- **(c)** L1 cache is slower than DRAM by a factor of 10.
- **(d)** DRAM takes 5 picoseconds, while L1 cache takes 1 second.

> **Correct Answer**: **(A)**
> **Bloom's Cognitive Level**: `Knowledge`
> **Architectural Rationale**: L1 cache operates at core clock speeds (~1 ns / ~4 cycles), whereas accessing DRAM requires traversing the memory controller and off-chip DDR bus (~60-80 ns / 200+ cycles), making cache hits over 50x faster.

Question 22

What is the standard hardware cache line size on modern x86_64 processors?

- **(a)** 4 bytes
- **(b)** 16 bytes
- **(c) [CORRECT]** 64 bytes
- **(d)** 4096 bytes

> **Correct Answer**: **(C)**
> **Bloom's Cognitive Level**: `Knowledge`
> **Architectural Rationale**: Virtually all modern x86_64 processors (Intel Xeon, AMD EPYC) fetch and invalidate data in contiguous 64-byte chunks called cache lines.

Question 23

What does SIMD stand for in computer architecture, and how does it benefit deep learning tensor computations?

- **(a) [CORRECT]** Single Instruction Multiple Data; it executes one operation (e.g., FMA) simultaneously across multiple data elements in wide vector registers.
- **(b)** Serial Input Multi Device; it allows one mouse to control several servers.
- **(c)** Synchronous Instruction Memory Dispatch; it compiles Python to assembly.
- **(d)** Shared Interface for Memory Devices; it formats hard drives.

> **Correct Answer**: **(A)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: SIMD (Single Instruction Multiple Data) enables a single machine instruction (such as AVX-512 `_mm512_fmadd_ps`) to perform simultaneous multiply-adds across an entire vector of floating-point values in parallel.

Question 24

How wide are Intel AVX-512 vector registers (ZMM0 to ZMM31), and how many 32-bit floating-point numbers can one register hold?

- **(a)** 128 bits wide, holding 4 floats.
- **(b)** 256 bits wide, holding 8 floats.
- **(c) [CORRECT]** 512 bits wide, holding 16 single-precision (FP32) floats.
- **(d)** 1024 bits wide, holding 64 floats.

> **Correct Answer**: **(C)**
> **Bloom's Cognitive Level**: `Knowledge`
> **Architectural Rationale**: Intel AVX-512 registers are 512 bits wide ($512/32 = 16$), capable of holding sixteen 32-bit single-precision floats or thirty-two 16-bit half-precision floats simultaneously.

Question 25

Why is the PCIe bus frequently a major performance bottleneck during deep learning training on discrete GPU or accelerator nodes?

- **(a) [CORRECT]** Because PCIe bandwidth (e.g., ~64 GB/s for PCIe Gen5 x16) is drastically lower than accelerator on-chip HBM bandwidth (up to 2000-3000 GB/s).
- **(b)** Because PCIe cannot transmit binary floating-point numbers.
- **(c)** Because PCIe operates strictly at 10 megabits per second.
- **(d)** Because the PCIe bus only operates when the operating system is shutting down.

> **Correct Answer**: **(A)**
> **Bloom's Cognitive Level**: `Analyze`
> **Architectural Rationale**: High Bandwidth Memory (HBM) on modern accelerators delivers 2 to 3 TB/s of bandwidth, whereas the host-to-device PCIe Gen5 x16 interface provides only ~64 GB/s, creating a massive throughput chasm if data must constantly travel over PCIe.

Question 26

In a Non-Uniform Memory Access (NUMA) multi-socket server, what is 'remote memory access latency'?

- **(a)** Accessing files from a remote cloud server in another country.
- **(b) [CORRECT]** The additional latency incurred when a CPU core accesses DRAM physically attached to a different CPU socket across the inter-socket interconnect (UPI/QPI/Infinity Fabric).
- **(c)** The time it takes to wake up a hard drive from power-saving mode.
- **(d)** Accessing registers in an adjacent thread.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: On multi-socket servers, each CPU socket has directly attached local memory channels. Accessing memory wired to another socket requires hops across Intel UPI or AMD Infinity Fabric, increasing latency by 1.5x-2.5x.

Question 27

What is High Bandwidth Memory (HBM2e / HBM3), and how is it physically packaged in modern HPC accelerators (e.g., Intel GPU Max, Gaudi, NVIDIA H100)?

- **(a)** Standard DDR5 DIMMs plugged into motherboard slots.
- **(b) [CORRECT]** Vertically stacked DRAM dies connected through Through-Silicon Vias (TSVs) placed directly on the same silicon interposer adjacent to the compute chip.
- **(c)** USB flash drives configured in a RAID-0 array.
- **(d)** Solid-state drives connected via SATA cables.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Knowledge`
> **Architectural Rationale**: HBM stacks memory dies 3D-vertically using Through-Silicon Vias (TSVs) and integrates them on an interposer directly beside the compute die with thousands of interconnect wires, achieving terabytes/sec of bandwidth at lower power.

Question 28

What is the purpose of an input data pipeline prefetching mechanism (e.g., 'dataset.prefetch(tf.data.AUTOTUNE)') in deep learning?

- **(a)** To download the next day's weather forecast.
- **(b) [CORRECT]** To overlap CPU data preprocessing (decoding, augmentation) for step $N+1$ with accelerator compute execution for step N, preventing compute starvation.
- **(c)** To convert the training labels from strings to integers.
- **(d)** To permanently store intermediate gradients on hard disk.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: Prefetching overlaps preprocessing and training. While the accelerator runs forward and backward passes on batch N, the CPU background threads prepare and load batch $N+1$, hiding I/O and pipeline latency.

Question 29

What is a 'memory leak' in Python deep learning code, and how does it typically manifest during long-running training loops?

- **(a)** Physical silicon melting due to high voltage.
- **(b) [CORRECT]** Tensors or Python objects accumulating references in global scope (e.g., appending loss tensors to a list without calling '.item()'), leading to gradual exhaustion of RAM and an Out-Of-Memory (OOM) crash.
- **(c)** The CPU cooling fans slowing down.
- **(d)** Model weights becoming NaN (Not-a-Number).

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: Appending full computational graph tensors to a Python list retains references to the entire backward execution graph in memory. Over hundreds of steps, this consumes gigabytes of heap until the process is killed with an OOM error.

Question 30

Why is 'data layout transformation' (e.g., converting NHWC to NCHW or vice versa) an expensive operation for GPUs and CPUs?

- **(a)** Because it requires training a new neural network.

- **(b) [CORRECT]** Because it performs a non-contiguous memory transposition that requires strided reads and writes across DRAM, which underutilizes memory bus bursts and thrashes cache lines.
- **(c)** Because operating systems do not allow 4D arrays.
- **(d)** Because it doubles the numerical value of all weights.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Analyze`
> **Architectural Rationale**: Permuting tensor dimensions requires moving every byte to a new memory address in a strided fashion, which cannot be coalesced efficiently by memory controllers, causing low effective bandwidth.

Question 31

In distributed deep learning, what is the core concept of 'Data Parallelism'?

- **(a)** Partitioning the neural network layers across different nodes so each node holds 2 layers.
- **(b) [CORRECT]** Replicating the entire model on every worker, assigning a distinct subset (shard) of the training dataset to each worker, and synchronizing gradients across workers at each step.
- **(c)** Running different machine learning frameworks on different servers.
- **(d)** Storing all dataset samples in a single SQLite database file.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: In Data Parallelism, each worker holds a complete replica of the model weights, processes a unique slice of the batch, and participates in an AllReduce operation to average gradients across all workers before updating weights.

Question 32

When is 'Model Parallelism' (Tensor Parallelism or Pipeline Parallelism) strictly required instead of pure Data Parallelism?

- **(a)** When the dataset has fewer than 100 images.
- **(b) [CORRECT]** When a single model's parameters and activation memory exceed the physical memory capacity of an individual accelerator.
- **(c)** When training on a laptop with only one CPU core.
- **(d)** When the learning rate is smaller than 0.001.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Analyze`
> **Architectural Rationale**: If a neural network has 70 billion parameters (requiring ~140 GB in FP16 for weights alone, plus optimizer states and activations), it cannot fit into a single 80 GB device. The model itself must be split across multiple devices.

Question 33

In the MPI standard, what does the collective operation 'MPI_Bcast' accomplish?

- **(a)** It sums arrays from all processes and stores the result on all processes.

- **(b)** It distributes equal chunks of a large array from a root process to all processes in the communicator.
- **(c) [CORRECT]** It broadcasts an identical buffer from a designated root process to all other processes in the communicator.
- **(d)** It shuts down all worker nodes immediately.

> **Correct Answer**: **(C)**
> **Bloom's Cognitive Level**: `Knowledge`
> **Architectural Rationale**: MPI_Bcast sends a copy of data buffer from the specified root rank to every rank in the communicator, ensuring all ranks possess identical data.

Question 34

What is the key functional difference between 'Reduce' and 'AllReduce' collective operations?

- **(a)** Reduce performs addition, while AllReduce performs subtraction.
- **(b) [CORRECT]** Reduce combines data across all processes using an operator (e.g., SUM) and delivers the result only to a single root rank; AllReduce delivers the final reduced result to ALL processes.
- **(c)** AllReduce only works on single-node computers.
- **(d)** Reduce runs in parallel, whereas AllReduce runs serially.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: MPI_Reduce leaves the aggregated result only on the root process. MPI_Allreduce computes the reduction and ensures every participating process receives a copy of the final result.

Question 35

What is the primary challenge of Asynchronous SGD (Async SGD) compared to Synchronous SGD in distributed training?

- **(a)** Async SGD cannot run on multiple machines.
- **(b) [CORRECT]** Workers apply parameter updates based on stale weights (the 'stale gradient problem'), which destabilizes convergence and degrades final model accuracy.
- **(c)** Async SGD requires all workers to pause and wait for the slowest worker at every step.
- **(d)** Async SGD requires 100 times more network bandwidth than Synchronous SGD.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Analyze`
> **Architectural Rationale**: In Async SGD, fast workers compute gradients on old weights while other workers update the central model. Applying gradients calculated on outdated model states causes noisy, divergent optimization trajectories.

Question 36

What is the 'straggler problem' in synchronous distributed deep learning training?

- **(a)** A bug where the compiler drops trailing zeros from floating point numbers.

- **(b) [CORRECT]** A single slow worker node (due to thermal throttling, hardware faults, or I/O stalls) delays the entire cluster because all other nodes must wait for it at the synchronization barrier.
- **(c)** Network packets arriving out of order over TCP.
- **(d)** Images having mismatched dimensions in the training set.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: Because Synchronous SGD enforces an AllReduce barrier at the end of each step, the step time of the entire cluster is dictated by the slowest worker (the straggler), leading to idle GPU compute wastage.

Question 37

Why is gradient bucketing (or gradient fusion) utilized in distributed training frameworks like Horovod and PyTorch DDP?

- **(a) [CORRECT]** To reduce network overhead by aggregating thousands of small layer gradient tensors into a few large contiguous memory buffers before launching AllReduce.
- **(b)** To sort the gradients in alphabetical order.
- **(c)** To convert the neural network into an image classification model.
- **(d)** To encrypt model weights before transmission.

> **Correct Answer**: **(A)**
> **Bloom's Cognitive Level**: `Analyze`
> **Architectural Rationale**: Launching individual AllReduce calls for hundreds of small weight tensors introduces immense network latency and software launch overhead. Bucketing copies gradients into contiguous buffers (e.g., 25MB) so network transfers achieve near-line-rate bandwidth.

Question 38

What is the difference between latency and bandwidth in a high-speed HPC network fabric?

- **(a)** Latency is the maximum volume of data transferred per second; bandwidth is the time taken for a bit to travel.
- **(b) [CORRECT]** Latency is the round-trip transit time for a small message (e.g., microseconds); bandwidth is the sustained data transfer rate (e.g., Gbps or GB/s).
- **(c)** Latency and bandwidth are identical physical quantities measured in Hertz.
- **(d)** Latency applies only to wireless networks, while bandwidth applies only to fiber optics.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Knowledge`
> **Architectural Rationale**: Network latency is the time delay for a packet to traverse the fabric from source to destination (measured in μs), whereas bandwidth is the maximum throughput of data the link can carry (GB/s).

Question 39

What is Remote Direct Memory Access (RDMA), and why is it essential for distributed deep learning over InfiniBand or RoCE?

- **(a)** A technique to remotely reboot hard drives over the internet.

- **(b) [CORRECT]** A hardware capability allowing network adapters to transfer data directly between the memories of two remote servers without involving either server's OS kernel or CPU.
- **(c)** An unmanaged protocol that converts IP packets into analog television signals.
- **(d)** A cloud backup service for Python source code.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: RDMA bypasses the operating system kernel and CPU entirely, reading and writing directly to physical application memory across the network with microsecond latency and zero CPU memory copy overhead.

Question 40

In a distributed cluster of 8 nodes with 8 GPUs each, what is the total 'world size' in Horovod or MPI terms?

- **(a)** 8
- **(b)** 16
- **(c) [CORRECT]** 64
- **(d)** 512

> **Correct Answer**: **(C)**
> **Bloom's Cognitive Level**: `Application`
> **Architectural Rationale**: The world size is the total number of parallel processes participating in the distributed job: $8 \text{ nodes} \times 8 \text{ GPUs/node} = 64 \text{ total processes (ranks)}$.

Question 41

What is the primary role of a Workload Manager (Job Scheduler) such as Slurm in an enterprise HPC cluster?

- **(a)** To replace Linux operating system kernels with Windows.
- **(b) [CORRECT]** To arbitrate access to shared compute resources among competing users, queue jobs, enforce fair-share policies, and monitor execution across compute nodes.
- **(c)** To edit Python code and fix syntax errors automatically.
- **(d)** To manage physical power cabling in the server racks.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: A workload manager queues user job requests, evaluates available CPU/GPU/memory resources, and allocates nodes according to scheduling priority and fair-share policies.

Question 42

In a Slurm cluster, what is the role of the 'slurmd' daemon running on each compute node?

- **(a)** It manages the central accounting database for all billing.
- **(b) [CORRECT]** It listens for job allocation orders from 'slurmctld', launches user tasks, monitors their CPU/memory consumption, and signals job completion.
- **(c)** It compiles TensorFlow from source code.

- **(d)** It provides a web GUI for submitting batch jobs.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Knowledge`
> **Architectural Rationale**: 'slurmd' is the node daemon executing on every compute node. It monitors node hardware, receives job step instructions from the central 'slurmctld', and manages local task execution.

Question 43

Which command is used to submit a non-interactive batch script to the Slurm workload manager?

- **(a)** srun --interactive
- **(b) [CORRECT]** sbatch job_script.slurm
- **(c)** scancel all
- **(d)** sinfo --kill

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Knowledge`
> **Architectural Rationale**: 'sbatch' takes a batch script containing `#SBATCH` directives, validates the submission, queues the job in Slurm, and returns a unique Job ID immediately.

Question 44

What is the difference between 'srun' and 'sbatch' in Slurm?

- **(a) [CORRECT]** 'sbatch' submits a job to the batch queue and returns immediately; 'srun' launches job steps interactively or concurrently and blocks until tasks finish.
- **(b)** 'sbatch' only runs Fortran code, while 'srun' only runs Python.
- **(c)** 'srun' is used to delete jobs from the queue.
- **(d)** 'sbatch' can only run on a single CPU core.

> **Correct Answer**: **(A)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: 'sbatch' is asynchronous for batch processing. 'srun' is used within batch scripts or directly from the terminal to launch parallel tasks across allocated nodes in real time.

Question 45

Which Slurm directive specifies the maximum allowable runtime for a job before it is terminated by the scheduler?

- **(a)** #SBATCH --mem=32G
- **(b) [CORRECT]** #SBATCH --time=02:00:00
- **(c)** #SBATCH --partition=compute
- **(d)** #SBATCH --output=slurm-%j.out

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Knowledge`
> **Architectural Rationale**: '#SBATCH --time=HH:MM:SS' sets a hard wall-clock limit. If the job runs longer than this specified duration, Slurm sends SIGTERM followed by SIGKILL to release the nodes.

Question 46

What does the symbol '%j' represent in the Slurm directive '#SBATCH --output=training_%j.log'?

- **(a)** The user's username.
- **(b)** The name of the partition.
- **(c) [CORRECT]** The unique numerical Job ID allocated by Slurm.
- **(d)** The current date and time.

> **Correct Answer**: **(C)**
> **Bloom's Cognitive Level**: `Application`
> **Architectural Rationale**: '%j' in Slurm filename templates is automatically substituted with the job's unique numerical Job ID (e.g., `training_124598.log`), preventing overlapping logs.

Question 47

What does the command 'sinfo' display in a Slurm cluster environment?

- **(a)** The real-time CPU temperature of the head node.
- **(b) [CORRECT]** The status of partitions and compute nodes in the cluster (e.g., idle, allocated, draining, down).
- **(c)** The remaining storage quota in the user's home directory.
- **(d)** The git commit history of the repository.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Knowledge`
> **Architectural Rationale**: 'sinfo' queries the Slurm controller to provide a high-level summary of partitions, node counts, and operational node states (idle, alloc, drain, down).

Question 48

Why is training deep learning models on a shared Network File System (NFS) containing millions of tiny image files discouraged on HPC clusters?

- **(a)** Because NFS does not support JPEG or PNG images.
- **(b) [CORRECT]** Because millions of tiny file open/read metadata operations overwhelm the centralized NFS metadata server, causing massive I/O stalls and bringing down filesystem responsiveness for all users.
- **(c)** Because NFS requires all files to be smaller than 10 bytes.
- **(d)** Because NFS automatically deletes files after 24 hours.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Analyze`
> **Architectural Rationale**: Reading millions of small files incurs severe metadata overhead (inodes, permissions, directory lookups) on standard NFS. HPC best practice is to pack datasets into sequential container formats (e.g., TFRecords, HDF5, WebDataset) stored on parallel filesystems like Lustre.

Question 49

What is the purpose of the 'Environment Modules' system (`module load ...`) widely deployed on HPC clusters?

- **(a)** To dynamically adjust the terminal screen brightness.
- **(b) [CORRECT]** To dynamically modify the user's environment (PATH, LD_LIBRARY_PATH, MANPATH) to load specific compiler versions, MPI runtimes, and software packages without version conflicts.
- **(c)** To reboot individual compute nodes without administrator permissions.
- **(d)** To encrypt Python scripts before execution.

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Understand`
> **Architectural Rationale**: Lmod / Environment Modules dynamically set and unset shell environment variables, allowing users to switch between different compiler suites (e.g., GCC vs Intel oneAPI) and libraries cleanly.

Question 50

If a user needs to cancel a queued or running Slurm job with Job ID 89452, which command should they execute?

- **(a)** sbatch --delete 89452
- **(b) [CORRECT]** scancel 89452
- **(c)** squeue --kill 89452
- **(d)** sinfo --remove 89452

> **Correct Answer**: **(B)**
> **Bloom's Cognitive Level**: `Application`
> **Architectural Rationale**: 'scancel ' signals Slurm to immediately revoke resource allocation and terminate all running processes associated with that specific Job ID.
